Desde cero hasta ChatGPT 🤖

Día 1: Redes neuronales y PyTorch

José Antonio Hernández López

2 de marzo de 2026

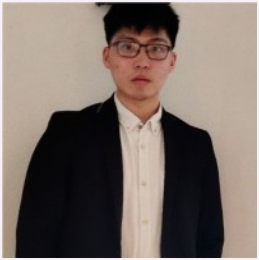
Sobre los docentes 👩‍🏫

José Antonio Hernández López

- Matemático e informático
- Doctor en Informática
- Profesor universitario de la UMU
- Investigación: LLMs para generación de código
- joseantonio.hernandez6@um.es

Ronghao Pan

- Ingeniero informático
- Doctorando e investigador en PLN
- Miembros del TECNOMOD y UMUTeam
- Investigación: LLMs, ASR, VLM
- ronghao.pan@um.es



Objetivo del curso 🎯

Estoy seguro de que muchos de vosotros sabéis

- Crear una red neuronal con, por ejemplo, Keras y entrenarla con un dataset
- Que una MLP son varias capas lineales con ReLU/sigmoid
- Instanciar distintas arquitecturas (MLP, CNN, RNN) según el problema
- Usar ChatGPT desde la UI
- Que un agente son LLMs *conectados* con herramientas externas

Objetivo del curso 🎯

Estoy seguro de que muchos de vosotros sabéis

- Crear una red neuronal con, por ejemplo, Keras y entrenarla con un dataset
- Que una MLP son varias capas lineales con ReLU/sigmoid
- Instanciar distintas arquitecturas (MLP, CNN, RNN) según el problema
- Usar ChatGPT desde la UI
- Que un agente son LLMs *conectados* con herramientas externas

pero... ¿sabéis...

- cómo funciona una red neuronal por dentro?
- de dónde viene la arquitectura transformer?
- cómo se ha llegado hasta ChatGPT? GPT vs ChatGPT?
- en qué se basan los agentes LLMs?

Objetivo del curso 🎯

Estoy seguro de que muchos de vosotros sabéis

- Crear una red neuronal con, por ejemplo, Keras y entrenarla con un dataset
- Que una MLP son varias capas lineales con ReLU/sigmoid
- Instanciar distintas arquitecturas (MLP, CNN, RNN) según el problema
- Usar ChatGPT desde la UI
- Que un agente son LLMs *conectados* con herramientas externas

pero... ¿sabéis...

- cómo funciona una red neuronal por dentro?
- de dónde viene la arquitectura transformer?
- cómo se ha llegado hasta ChatGPT? GPT vs ChatGPT?
- en qué se basan los agentes LLMs?

El **objetivo del curso** es que comprendáis en profundidad el funcionamiento interno de las redes neuronales, yendo un paso más allá de lo que suele encontrarse en tutoriales o recursos generales de Internet. Seguiremos un enfoque progresivo, en el que iremos construyendo poco a poco todo el conocimiento necesario sobre redes neuronales hasta llegar a entender cómo funcionan modelos como ChatGPT.

Roadmap general 🗺️

1. Redes neuronales y Pytorch - **Día 1**
2. Redes recurrentes y tokenización - **Día 2**
3. Mecanismo de atención, Transformers, Pre-training y Fine-tuning - **Día 3**
4. Modelos decoder-only. De GPT a ChatGPT - **Día 4.1**
5. Agentes LLM - **Día 4.2**

Roadmap del día 1 🗺️

1. ¿Qué son las redes neuronales?
   - Definición de red y arquitecturas notables
   - Redes y entrenamiento
2. ¿Cómo se entrenan las redes neuronales?
   - 1º Ingrediente - Gradiente descendente
   - 2º Ingrediente - Backpropagation
   - 3º Ingrendiente - Stochastic Gradient Descent
   - Lo cocinamos todo - Algoritmo de entrenamiento
3. Mejoras actuales en redes
   - Técnicas para evitar overfitting
   - Optimizadores
4. Pytorch
   - Conceptos básicos
   - Definición de redes
   - Bucle de entrenamiento

¿Qué son las redes neuronales?

Definición de red y arquitecturas notables

Redes neuronales

Red neuronal

Una red neuronal no es más que una función $f_\theta : \mathbb{R}^n \to \mathbb{R}^m$ diferenciable en casi todo punto que

- Recibe como **entrada** un vector x real $n-$dimensional
- Da como **salida** un vector $y = f(x)$ $m-$dimensional
- Depende de unos **parámetros o pesos** θ que deben calcularse entrenando
- El número de parámetros $|\theta|$ pueden ser miles, millones o billones
- En función de la forma de f_θ, tenemos varios tipos de redes o arquitecturas: MLP, CNN, RNN, transformer, etc.

Redes neuronales

Red neuronal

Una red neuronal no es más que una función $f_\theta : \mathbb{R}^n \to \mathbb{R}^m$ diferenciable en casi todo punto que

- Recibe como **entrada** un vector x real $n-$dimensional
- Da como **salida** un vector $y = f(x)$ $m-$dimensional
- Depende de unos **parámetros o pesos** θ que deben calcularse entrenando
- El número de parámetros $|\theta|$ pueden ser miles, millones o billones
- En función de la forma de f_θ, tenemos varios tipos de redes o arquitecturas: MLP, CNN, RNN, transformer, etc.

Por ejemplo (Iris dataset):

- Queremos un sistema que dado unas medidas de pétalo y sépalo sepan clasificar la flor en tres tipos: versicolor, setosa y virginica
- Entrada: vector de cuatro dimensiones con las medidas
- Salida: vector de tres dimensiones con las probabilidades

MLP (Multi-Layer Perceptron)

Idea

Una MLP es una red que trata la entrada como un **vector plano**.

- Entrada: $x \in \mathbb{R}^n$
- Salida: $y \in \mathbb{R}^m$
- Compuesta por capas totalmente conectadas

$$f_\theta(x) = \phi_L(W_L\, \phi_{L-1}(\dots \phi_1(W_1 x + b_1) \dots) + b_L)$$

Características

- L es el número de capas
- Las ϕ_i son funciones de activación, no lineales: tanh, ReLU, σ, etc.
- Los parámetros (θ) son las matrices W_i y los sesgos b_i

RNN (Recurrent Neural Network)

Idea

Una RNN procesa **secuencias**, reutilizando los mismos parámetros en cada paso.

- Entrada: secuencia $(x_1, x_2, \ldots, x_T)$, con $x_t \in \mathbb{R}^n$
- Estado oculto: $h_t \in \mathbb{R}^d$

$$h_t = \phi(W_h h_{t-1} + W_x x_t + b)$$
$$y_t = g(W_y h_t)$$

Características

- Comparte parámetros a lo largo del tiempo: W_h, W_x y W_y
- ϕ es alguna función de activación
- g es la función que convierte el estado oculto en la salida observable
- Hay varias arquitecturas, las más conocidas: LSTM y GRU
- Captura dependencias temporales

CNN (Convolutional Neural Network)

Idea

Una CNN explota la **estructura espacial local** de los datos (imágenes, señales, etc.).

- Entrada: tensor $x \in \mathbb{R}^{H \times W \times C}$
- Filtros locales compartidos

$$(\mathbf{K} * \mathbf{x})(i, j) = \sum_{u,v} \mathbf{K}(u, v)\, \mathbf{x}(i - u, j - v)$$

Características

- Los parámetros son $\mathbf{K}(u, v)$ y suele ser una matriz pequeña
- Mucho menos parámetros que una MLP
- Ideal para imágenes y datos espaciales

Transformer

Idea

Un Transformer procesa **secuencias completas en paralelo** usando mecanismos de **self-attention**, sin recurrencia ni convolución.

- Entrada: secuencia $(x_1, \ldots, x_T)$, con $x_t \in \mathbb{R}^d$
- Se añade información de posición (positional encoding)

Self-Attention

$$\mathrm{Att}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Características

- No usa recurrencia ni convolución
- Captura dependencias largas de forma eficiente
- Totalmente paralelizable

Takeaways

Takeaways

- Las redes no son más que funciones $f_\theta : \mathbb{R}^n \to \mathbb{R}^m$ que mapean vectores a otros vectores
- Las redes dependen de unos parámetros θ que deben obtenerse entrenando la red
- El número de parámetros pueden ser miles, millones o billones
- Hay muchísimos tipos de redes/definiciones de f_θ: MLPs, RNNs, CNNs, transformers, etc.

Takeaways

Takeaways

- Las redes no son más que funciones $f_\theta : \mathbb{R}^n \to \mathbb{R}^m$ que mapean vectores a otros vectores
- Las redes dependen de unos parámetros θ que deben obtenerse entrenando la red
- El número de parámetros pueden ser miles, millones o billones
- Hay muchísimos tipos de redes/definiciones de f_θ: MLPs, RNNs, CNNs, transformers, etc.

- Hay más *tipos* de redes: GNNs, autoencoders, etc.
- En este curso vamos a ver: MLPs, RNNs y transformers
- Los modelos más populares están basados en la arquitectura transfomer (ChatGPT, Gemini, DeepSeek, etc.)

Redes y entrenamiento

Redes neuronales y entrenamiento

Todas las redes neuronales dependen de unos pesos (θ) que deben ser entrenados para que la red sea útil. Es decir...

Redes neuronales y entrenamiento

Todas las redes neuronales dependen de unos pesos (θ) que deben ser entrenados para que la red sea útil. Es decir...

Todas las redes pasan por un proceso de entrenamiento en el que los parámetros θ se calculan.

Redes neuronales y entrenamiento

Todas las redes neuronales dependen de unos pesos (θ) que deben ser entrenados para que la red sea útil. Es decir...

Todas las redes pasan por un proceso de entrenamiento en el que los parámetros θ se calculan.

¿Cómo se hace eso?

- Normalemente se dispone de un conjunto de entrenamiento $T = \{(x_1, y_1), \ldots, (x_n, y_n)\}$ (conjunto de entradas con salidas esperadas).

Redes neuronales y entrenamiento

Todas las redes neuronales dependen de unos pesos (θ) que deben ser entrenados para que la red sea útil. Es decir...

Todas las redes pasan por un proceso de entrenamiento en el que los parámetros θ se calculan.

¿Cómo se hace eso?

- Normalemente se dispone de un conjunto de entrenamiento $T = \{(x_1, y_1), \ldots, (x_n, y_n)\}$ (conjunto de entradas con salidas esperadas).
- La idea es que los θ deben escogerse de manera que f_θ lo haga bien en T con la esperanza de que generalice.

Redes neuronales y entrenamiento

Todas las redes neuronales dependen de unos pesos (θ) que deben ser entrenados para que la red sea útil. Es decir...

Todas las redes pasan por un proceso de entrenamiento en el que los parámetros θ se calculan.

¿Cómo se hace eso?

- Normalemente se dispone de un conjunto de entrenamiento $T = \{(x_1, y_1), \ldots, (x_n, y_n)\}$ (conjunto de entradas con salidas esperadas).
- La idea es que los θ deben escogerse de manera que f_θ lo haga bien en T con la esperanza de que generalice.
- Esto se traduce en encontrar θ tal que se minice está **función de pérdida o loss**:

$$L(\theta) = \sum_{i=1}^{n} E(f_\theta(x_i), y_i)$$

donde E es el error de la salida de la red $f_\theta(x_i)$ y la salida esperada y_i.

¿Cómo se entrenan las redes neuronales?

1º Ingrediente - Gradiente descendente

Optimización de funciones

Supongamos que tenemos la función $L(\theta) = \theta^2$ y queremos encontrar el valor de θ que minimiza dicha función.
¿Cómo se haría?

Optimización de funciones

Supongamos que tenemos la función $L(\theta) = \theta^2$ y queremos encontrar el valor de θ que minimiza dicha función.
¿Cómo se haría? Cómo nos enseñaron en el instituto 😆

- Calculamos la derivada de la función $L'(\theta) = 2\theta$
- Un mínimo (relativo) está donde $L'(\theta) = 0 \Rightarrow \theta = 0$
- Vemos si es mínimo calculando la segunda derivada y viendo si es positiva

En este caso el mínimo relativo coincide con el absoluto (no siempre es así). En la práctica (en redes neuronales), nos conformamos con uno relativo que sea bueno.

Optimización de funciones

Vamos a subir la complejidad un poco. Supongamos que tenemos dos parámetros, θ_1 y θ_2, y la función $L(\theta_1, \theta_2) = \theta_1^2 + \theta_2^2$. Queremos encontrar el punto $(\hat{\theta}_1, \hat{\theta}_2)$ que minimize L. Aunque es algo más complejo (función de dos dimensiones), el procedimiento es similar:

- Calculamos la derivada de la función. Aquí la *derivada* es el gradiente:

$$\nabla L(\theta_1, \theta_2) = \left(\frac{\partial L}{\partial \theta_1}, \frac{\partial L}{\partial \theta_2} \right) = (2\theta_1, 2\theta_2)$$

Optimización de funciones

Vamos a subir la complejidad un poco. Supongamos que tenemos dos parámetros, θ_1 y θ_2, y la función $L(\theta_1, \theta_2) = \theta_1^2 + \theta_2^2$. Queremos encontrar el punto $(\hat{\theta}_1, \hat{\theta}_2)$ que minimize L. Aunque es algo más complejo (función de dos dimensiones), el procedimiento es similar:

- Calculamos la derivada de la función. Aquí la *derivada* es el gradiente:

$$\nabla L(\theta_1, \theta_2) = \left(\frac{\partial L}{\partial \theta_1}, \frac{\partial L}{\partial \theta_2} \right) = (2\theta_1, 2\theta_2)$$

- Buscamos los puntos donde el gradiente es cero: $\nabla L(\theta_1, \theta_2) = 0 \Rightarrow (\theta_1, \theta_2) = (0, 0)$. Por suerte solo hay una solución 😅.

Optimización de funciones

Vamos a subir la complejidad un poco. Supongamos que tenemos dos parámetros, θ_1 y θ_2, y la función $L(\theta_1, \theta_2) = \theta_1^2 + \theta_2^2$. Queremos encontrar el punto $(\hat{\theta}_1, \hat{\theta}_2)$ que minimize L. Aunque es algo más complejo (función de dos dimensiones), el procedimiento es similar:

- Calculamos la derivada de la función. Aquí la *derivada* es el gradiente:

$$\nabla L(\theta_1, \theta_2) = \left(\frac{\partial L}{\partial \theta_1}, \frac{\partial L}{\partial \theta_2} \right) = (2\theta_1, 2\theta_2)$$

- Buscamos los puntos donde el gradiente es cero: $\nabla L(\theta_1, \theta_2) = 0 \Rightarrow (\theta_1, \theta_2) = (0, 0)$. Por suerte solo hay una solución 😅.
- Comprobamos que es un mínimo viendo si tiene segunda derivada positiva. Aquí la derivada segunda es una matriz Hessiana 2×2. La matriz tiene que ser definida positiva.

$$H(L)(\theta_1, \theta_2) = \begin{bmatrix} \frac{\partial^2 L}{\partial \theta_1^2} & \frac{\partial^2 L}{\partial \theta_1 \partial \theta_2} \\ \frac{\partial^2 L}{\partial \theta_2 \partial \theta_1} & \frac{\partial^2 L}{\partial \theta_2^2} \end{bmatrix}$$

Optimización de funciones

Takeaway principal 1º parte

Si tenemos que optimizar una función de K parámetros como *nos enseñaron en el instituto* hay que:

- Calcular la derivada (gradiente) e igualarlo a cero: resolver un sistema de K ecuaciones potencialmente no lineales.
- Estudiar la segunda derivada (Hessiano): matriz de $K \times K$.

Optimización de funciones

Takeaway principal 1º parte

Si tenemos que optimizar una función de K parámetros como *nos enseñaron en el instituto* hay que:

- Calcular la derivada (gradiente) e igualarlo a cero: resolver un sistema de K ecuaciones potencialmente no lineales.
- Estudiar la segunda derivada (Hessiano): matriz de $K \times K$.

Como veremos más adelante, entrenar ChatGPT es esquivalente a minimizar una función $L(\theta_1, \ldots, \theta_{10^{12}})$ 😅. Por tanto:

Takeaway principal 2º parte

El método que *nos enseñaron en el instituto* no funciona porque:

- Los métodos numéricos de resolución de ecuaciones no funcionan bien en altas dimensiones.
- Las arquitecturas de los ordenadores actuales hacen imposible manejar una matriz $10^{12} \times 10^{12}$.

Gradiente descendente al rescate

Idea principal:

- Partimos de un valor inicial aletorio de $\theta_0 \in \mathbb{R}^K$
- Mientras no hayamos convergido:
  1. Calculamos la dirección de descenso en ese punto
  2. Actualizamos el parámetro en esa dirección

Esto no garantiza encontrar el mínimo global, pero en la práctica funciona bien.

Gradiente descendente al rescate

¿Cuál es la dirección de descenso?

Gradiente descendente al rescate

¿Cuál es la dirección de descenso?

Propiedad del gradiente

El gradiente ∇L siempre apunta en la dirección de mayor subida, por lo que su negativo $-\nabla L$ apunta en la dirección de mayor bajada.

Por tanto es razonable hacer, en cada paso,

$$\theta \leftarrow \theta - \lambda \nabla L(\theta)$$

donde a $\lambda > 0$ se le llama **learning rate** y suele ser un valor pequeño (ej. 0.01).

Gradiente descendente al rescate

¿Cuál es la dirección de descenso?

Propiedad del gradiente

El gradiente ∇L siempre apunta en la dirección de mayor subida, por lo que su negativo $-\nabla L$ apunta en la dirección de mayor bajada.

Por tanto es razonable hacer, en cada paso,

$$\theta \leftarrow \theta - \lambda \nabla L(\theta)$$

donde a $\lambda > 0$ se le llama **learning rate** y suele ser un valor pequeño (ej. 0.01).
¿Cuándo paramos el bucle?

Gradiente descendente al rescate

¿Cuál es la dirección de descenso?

Propiedad del gradiente

El gradiente ∇L siempre apunta en la dirección de mayor subida, por lo que su negativo $-\nabla L$ apunta en la dirección de mayor bajada.

Por tanto es razonable hacer, en cada paso,

$$\theta \leftarrow \theta - \lambda \nabla L(\theta)$$

donde a $\lambda > 0$ se le llama **learning rate** y suele ser un valor pequeño (ej. 0.01).
¿Cuándo paramos el bucle?
Cuando $||\nabla L(\theta)||$ es suficientemente pequeño o hemos alcanzado un número máximo de iteraciones (suele ser lo segundo).

Gradiente descendente al rescate

Gradiente descendente

1. $\theta \leftarrow \theta_0 \in \mathbb{R}^K$ aleatorio (se suele inicializar a valores pequeños)
2. mientras $||\nabla L(\theta)|| >> 0$ o no hayamos superado el máximo de iteraciones:

$$\theta \leftarrow \theta - \lambda \nabla L(\theta)$$

Gradiente descendente en redes neuronales

Pero, espera un segundo... para entrenar la red tengo que encontrar el θ que minice un función derivable (de la que podemos sacar el gradiente)

$$L(\theta) = \sum_{i=1}^{n} E(f_\theta(x_i), y_i)$$

Eso sé hacerlo ☺️ $\longrightarrow$ gradiente descendente

Gradiente descendente en redes neuronales

Takeaways

- Una red neuronal es una función f_θ que procesa *cualquier cosa* y da como resultado *cualquier cosa*
- Para escoger θ hay que pasar un proceso de entrenamiento
- El proceso de entrenamiento es minimizar

$$L(\theta) = \sum_{i=1}^{n} E(f_\theta(x_i), y_i)$$

  usando el conjunto de entrenamiento $T = \{(x_1, y_1), \ldots, (x_n, y_n)\}$
- La función se minimiza con el gradiente descendente

Ejercicios

Veamos ejemplos en el Colab.

- $L(\theta) = \theta^2$
- $L(\theta_1, \theta_2) = \theta_1^2 + \theta_2^2$
- $L(\theta_1, \ldots, \theta_{100}) = \sum_{i=1}^{100} (\theta_i - 1)$
- Regresión lineal

2º Ingrediente - Backpropagation

Running example

Supongamos que queremos resolver el problema de clasificación de flores: iris versicolor, setosa y virgínica (Iris dataset).

- La entrada tiene 4 dimensiones x: longitud y anchura del sépalo y el pétalo
- La salida tiene 3 dimensiones y: la probabilidad de que es versicolor, la probabilidad de que es setosa y la probabilidad de que es virgínica
- El conjunto de entrenamiento tiene 150 ejemplares $|T| = 150$
- Cada ejemplo de entrenamiento es (x, y) donde x son las longitudes y anchuras e y es un vector one-hot que indica la clase (cada clase tiene asociada una dimensión)

Running example

Para ello, vamos a definir una red de esta manera:

$$f_{W,b}(x) = \text{softmax}(Wx + b),$$

softmax se define como $p_j = \frac{e^{z_j}}{\sum_{k=1}^{3} e^{z_k}}$

- El gradiente descendente se aplica a $L(\theta) = \sum_{i=1}^{150} E(f_\theta(x_i), y_i)$
- Al *error* $E(f(x), y) = -\sum_i y_i \log f(x)_i$ se le llama cross-entropy
- Los parámetros son $W_{3\times 4}$ y $b_{3\times 1}$. Total: 15 parámetros (tenemos que encontrar el mínimo de una función de 15 dimensiones)
- $\theta = (\theta_1, \ldots, \theta_{15}) = \{W, b\}$
- $\nabla L(\theta) = (\partial L/\partial\theta_1, \ldots, \partial L/\partial\theta_{15}) = \{\nabla_W L, \nabla_b L\}$

Backpropagation

El *backpropagation* es el procedimiento **que permite calcular (de manera eficiente) los gradientes de una función de pérdida respecto a todos los parámetros de una red neuronal**. Se basa en representar el cálculo como un grafo computacional dirigido (*computational graph*).

- **PyTorch** y Tensorflow son librerías que automatizan el *backpropagation*
- En estos frameworks, al definir una red, se construye un grafo computacional por debajo

Backpropagation: ejemplo

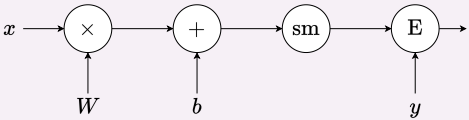
Imagínate que implementamos la siguiente red usando un framework de deep learning

$$f_{W,b}(x) = \text{softmax}(Wx + b)$$

donde

- Los parámetros son $W_{3\times 4}$ y $b_{3\times 1}$
- softmax se define como $p_j = \frac{e^{z_j}}{\sum_{k=1}^{3} e^{z_k}}$
- La función loss se le llama cross-entropy $E(f(x), y) = -\sum_i y_i \log f(x)_i$

El grafo resultante podría tener la siguiente pinta[1]:

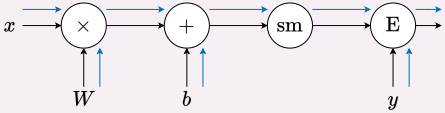


[1] E y sm se suelen hacer a la vez, en un mismo nodo

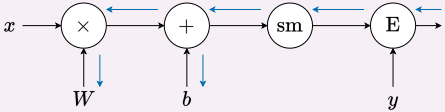
Backpropagation: ejemplo

Para calcular los gradientes, se hacen dos pasos

1. Forward: calculo el error $E(f(x), y)$



2. Backward: backpropago el error para calcular los grandientes para todos los parámetros ∇_W y ∇_b.



Sólo se calculan y guardan los gradientes (solo se backpropaga) en los nodos hoja que requieran gradiente (los parámetros).

Backpropagation: ejemplo

¿Qué ocurre dentro de cada nodo?

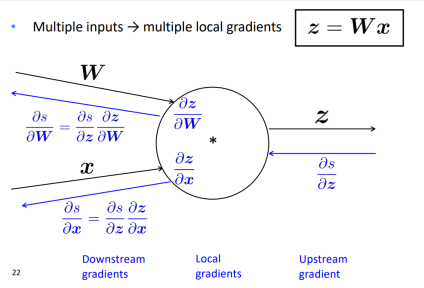


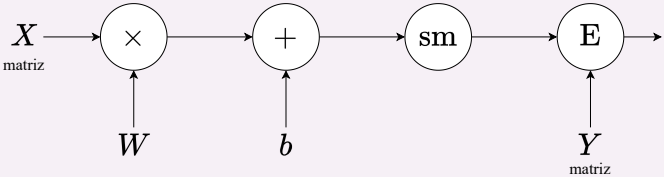
Figura: Figura extraída del curso CS224n Stanford

La idea principal es que dentro de un nodo se calcula como un *gradiente local* y, usando el gradiente entrante, se calculan los gradientes salientes. Esto efectivamente acabará calculando los gradientes con respecto a W y b gracias a la regla de la cadena...

Backpropagation: ejemplo

- En el ejemplo anterior estamos calculando el gradiente para un ejemplo $E(f(x), y)$. Pero el loss se calcula para todo el dataset (recordad que $L(\theta) = \sum_{i=1}^{n} E(f_\theta(x_i), y_i)$)
- En la práctica (lo que PyTorch/Tensorflow hace) lo que se hace es el forward/backward con varios ejemplos a la vez. Este conjunto de ejemplos se le llama *batch*
- En el gradiente descendente puro, el batch es igual al número de ejemplos del training

En otras palabras $f_{W,b}(X) = \text{softmax}(XW + \mathbf{1}b^\top)$



Backpropagation: conclusiones

Takeaways

- Cuando defines una red neuronal usando la PyTorch o Tensorflow, se construye un grafo computacional que no tiene ciclos
- A la hora de calcular los gradientes, se hace un 1) forward siguiendo el orden topológico del grafo y 2) un backward para calcular los gradientes (siguiendo el orden topológico inverso)
- En cada nodo del grafo, se aplica la regla de la cadena

3º Ingrendiente - Stochastic Gradient Descent

Stochastic Gradient Descent: motivación

Entrenar una red es, dado un conjunto de entrenamiento $T = \{(x_1, y_1), \ldots, (x_n, y_n)\}$, tenemos que encontrar los parámetros $\hat{\theta}$ que minimice

$$L(\theta) = \sum_{i=1}^{n} E(f_\theta(x_i), y_i).$$

Si aplicamos el grandiente descendente puro:

1. $\theta \leftarrow \theta_0 \in \mathbb{R}^K$
2. mientras no hayamos superado el máximo de iteraciones:

$$\theta \leftarrow \theta - \lambda \nabla L(\theta) \text{ haciendo backpropagation}$$

entonces cada actualización de θ usa todo el training dataset...

- muy caro y no escalable: hay que manejar una matriz muy grande X en la backpropgación
- converge lentamente en la práctica

Stochastic Gradient Descent: idea clave

En cada iteración no usamos todo el conjunto de entrenamiento T, sino un subconjunto (batch) $B \subset T$ con $|B| \ll |T|$.

- Calculamos la **loss del batch** usando solo las muestras de B.
- Actualizamos los parámetros usando **solo ese batch**.

$$\theta \leftarrow \theta - \lambda \nabla L_B(\theta)$$

Idea fundamental

Aunque cada batch es solo una pequeña parte del dataset, **al ir cambiando los batches**, la loss del batch

$$L_B(\theta)$$

se aproxima a la loss total del entrenamiento

$$L(\theta).$$

Stochastic Gradient Descent: ejemplo

Siguiendo con el ejemplo del Iris dataset. Si usáramos el gradiente descendente puro, en cada iteración:

- Hacemos un forward y un backward con 150 elementos para calcular los gradientes
- Actualizamos los parámetros

Para una sola actualización de pesos necesitamos ver 150 ejemplos. Sin embargo, con el SGD:

- Mezclamos el training dataset y lo dividimos en batches de tamaño 16
- Para cada batch
  - Hacemos un forward y un backward con 16 elementos
  - Actualizamos los parámetros

Los 150 ejemplos provocan 10 actualizaciones (9 batches de 16 elementos y uno de 6). Convergemos más rápido.

Lo cocinamos todo - Algoritmo de entrenamiento

Juntamos todos los ingredientes... 👩‍🍳

Ya tenemos todos los ingredientes para tener el algoritmo final de entrenamiento de redes.

1. ❤️ Grandiente descendente
2. 💛 Backpropagation
3. 💙 Stochastic Gradient Descent

Algoritmo final:

1. Inicializar los parámetros θ de la red ❤️
2. Repetir durante un número de **epochs**: ❤️ 💙
   - 2.1 Barajar el conjunto de entrenamiento 💙
   - 2.2 Dividir el dataset en **batches** 💙
   - 2.3 Para cada batch B: 💙
     - 2.3.1 Forward pass: calcular la salida de la red y el loss 💛
     - 2.3.2 Backward pass: gradientes 💛
     - 2.3.3 Actualizar los parámetros: ❤️ 💙

$$\theta \leftarrow \theta - \lambda \nabla L_B(\theta).$$

Entrenamiento final

Entrenar una red es, dado un conjunto de entrenamiento $T = \{(x_1, y_1), \ldots, (x_n, y_n)\}$, tenemos que encontrar los parámetros $\hat{\theta}$ que minimice

$$L(\theta) = \sum_{i=1}^{n} E(f_\theta(x_i), y_i).$$

Que minimice los errores del entrenamiento con la esperanza de que tenga muy poco error en los ejemplos no vistos.

Algoritmo final:

1. Inicializar los parámetros θ de la red. ❤️
2. Repetir durante un número de **epochs**: ❤️ 💙
   - 2.1 Barajar el conjunto de entrenamiento. 💙
   - 2.2 Dividir el dataset en **batches** 💙
   - 2.3 Para cada batch B: 💙
     - 2.3.1 Forward pass: calcular la salida de la red y el loss 💛
     - 2.3.2 Backward pass: gradientes
     - 2.3.3 Actualizar los parámetros ❤️ 💙

Entrenamiento final: ejemplo

Siguiendo con el ejemplo de Iris:

1. Inicializar los parámetros W y b
2. Repetir durante un número de **epochs** (por ejemplo, 100):
   - 2.1 Barajar el conjunto de entrenamiento
   - 2.2 Dividir el dataset en **batches** de 16 elementos
   - 2.3 Para cada batch B:
     - 2.3.1 Forward pass: calcular la loss del batch[2]:

$$Z_B = X_B W + \mathbf{1} b^\top, \qquad P_B = \text{softmax}(Z_B)$$

$$L_B = \frac{1}{16} \sum_{i=1}^{16} E(f_\theta(x_i), y_i)$$

     - 2.3.2 Backward pass: calcular

$$\nabla_{W,B} L, \quad \nabla_{b,B} L$$

     - 2.3.3 Actualizar los parámetros:

$$W \leftarrow W - \lambda \nabla_{W,B} L, \qquad b \leftarrow b - \lambda \nabla_{b,B} L$$

[2] se suele normalizar por el tamaño del batch. $1/|B|$ no cambia el punto óptimo de la función objetivo

Ejercicios

Veamos un ejemplo en el Colab con PyTorch:

- Iris dataset
- Definición simple de la red
- Grafo computacional
- SGD
- Bucle de entrenamiento
- Ejercicio: train/test split

Mejoras actuales en redes

Técnicas para evitar overfitting

¿Qué es el overfitting?

Overfitting

El overfitting (o sobreajuste) ocurre cuando una red neuronal aprende tan bien los datos de entrenamiento que memoriza en lugar de aprender las tendencias generales, resultando en un rendimiento excelente en datos conocidos pero muy pobre al enfrentarse a datos nuevos y desconocidos.

- El overfitting es un problema fundamental del machine learning en general
- Si te descuidas... Las redes, al tener muchos parámetros, van a memorizar los datos
- El overfitting afecta mucho a las redes
- La razón: *bias-variance trade-off*
- Mecanismos para evitarla: early stopping/monitorización del loss de validación, regularizaciones L_1, L_2 y dropout

¿Qué es el overfitting?

Ejemplo de overfitting:

- Problema de clasificación de imágenes
- Dataset imágenes $n = 200$
- Red CNN con 500M de parámetros

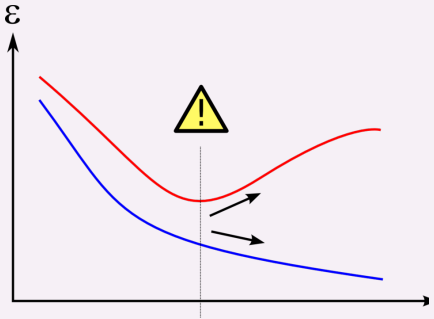


Figura: En azul, el error de entrenamiento; en rojo, el de ejemplos no vistos.

Mecanismo I: Early stopping

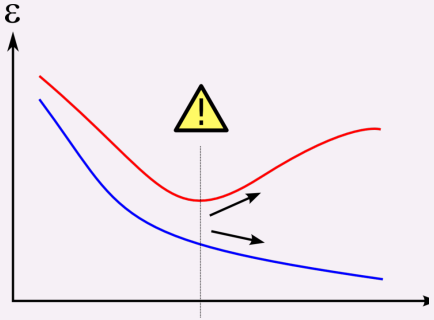
Ideas principales

- Llevar un conjunto de *validación* (con datos no vistos) y, en cada paso, después de actualizar los pesos, medir el error con ese conjunto
- Si el error es más pequeño que el mejor anterior, guardar un *checkpoint* del modelo
- En caso contrario y si se nos ha acabado la *paciencia* (número de actualizaciones sin mejorar el error), terminamos el entrenamiento.

Esta técnica es muy intutiva y fácil de implementar.

Mecanismo I: Early stopping

Intuitivamente, sería equivalente a parar cuando el error de validación empieza a subir y tiene pinta de no va a bajar. Nos quedamos con el modelo de menor error de validación encontrado.



Mecanismo I: Early stopping 💜

1. Inicializar los parámetros θ de la red
2. Inicializar los contadores de **early stopping** (mejor loss de validación, paciencia)
3. Repetir durante un número máximo de **epochs**:
   - 3.1 Barajar el conjunto de entrenamiento
   - 3.2 Dividir el dataset en **batches**
   - 3.3 Para cada batch B:
     - 3.3.1 Forward pass: calcular la salida de la red
     - 3.3.2 Calcular la loss del batch
     - 3.3.3 Backward pass
     - 3.3.4 Actualizar los parámetros θ
   - 3.4 Evaluar la **loss de validación** 💜
   - 3.5 Si la loss de validación mejora a la última mejor: 💜
     - 3.5.1 Guardar el modelo 💜
     - 3.5.2 Reiniciar el contador de paciencia 💜
   - 3.6 En caso contrario: 💜
     - 3.6.1 Incrementar el contador de paciencia 💜
     - 3.6.2 Si se supera la paciencia, **detener el entrenamiento (early stopping)** 💜
4. Cargar el último modelo guardado 💜

Mecanismo I: Monitorización del loss de validación

Algunas veces se sigue un criterio más simple:

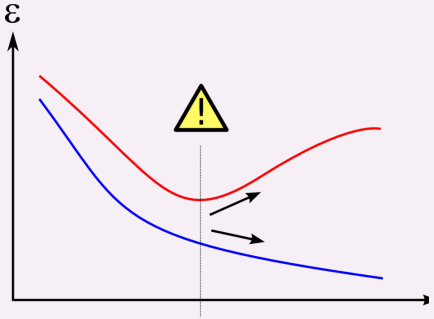
- Se evalua el loss de validación cada cierto tiempo (por ejemplo, al final de cada epoch)
- Si el loss de validación es mejor que el mejor anterior, se guarda el modelo
- Finalmente, se coge el modelo con el menor loss de validación observado durante el entrenamiento

Importante

En los entrenamientos de redes, siempre se lleva un conjunto de validación para monitorizar el loss de validación.

Mecanismo I: Monitorización del loss de validación

Intuitivamente, sería equivalente a pintar toda la gráfica y quedarnos con el modelo que minimiza el loss de validación.



Mecanismo I: Monitorización del loss de validación 💜

1. Inicializar los parámetros θ de la red
2. Inicializar los contadores de **early stopping** (mejor loss de validación, paciencia).
3. Repetir durante un número máximo de **epochs**:
   - 3.1 Barajar el conjunto de entrenamiento
   - 3.2 Dividir el dataset en **batches**
   - 3.3 Para cada batch B:
     - 3.3.1 Forward pass: calcular la salida de la red
     - 3.3.2 Calcular la loss del batch
     - 3.3.3 Backward pass
     - 3.3.4 Actualizar los parámetros θ
   - 3.4 Evaluar la **loss de validación** 💜
   - 3.5 Si la loss de validación mejora a la última mejor: 💜
     - 3.5.1 Guardar el modelo. 💜
4. Cargar el último modelo guardado 💜

Mecanismo II: Regularización L_i

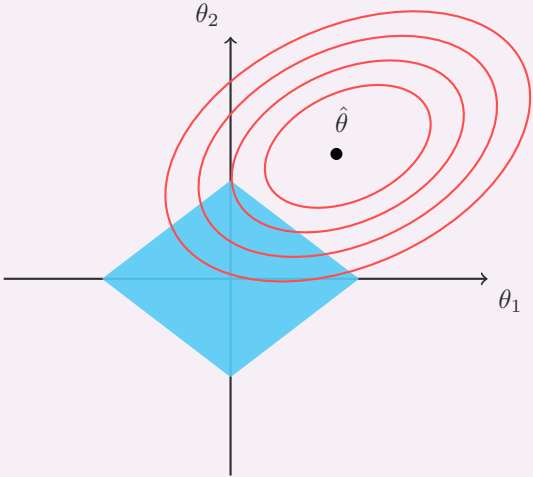
Este mecanismo de regularización añade un término a la función de pérdida:

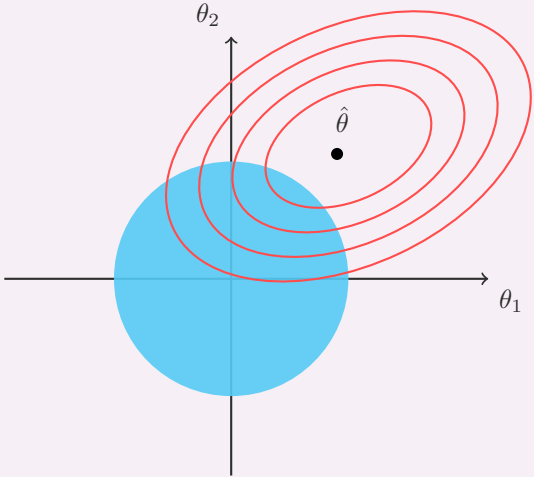
$$L(\theta) = \sum_{i=1}^{n} E(f_\theta(x_i), y_i) + \alpha \|\theta\|_i^i$$

con $i = 1, 2$. α controla cuanto se penaliza y suele estar entre $0.01 - 0.1$

- La que más se usa en deep learning es $i = 2$ (llamada usualmente *Ridge* o *weight decay*)
- La idea intuitiva es que, al minimizar eso, estamos forzando a que $\theta \approx 0$ y, por consiguiente, evitamos que el modelo aprenda de los datos

Mecanismo II: Regularización L_i





Mecanismo III: Dropout

Idea principal

Durante el entrenamiento, apagamos aleatoriamente salidas de una capa en cada forward pass (los hacemos 0). De este modo, se evita que la red se ajuste mucho a los datos.

- Sea $\mathbf{h} = (h_1, h_2, \ldots, h_d)$ la salida de una capa
- Durante dropout, definimos una máscara aleatoria (de 0s y 1s):

$$\mathbf{m} = (m_1, \ldots, m_d), \quad m_i \sim \text{Bernoulli}(p)$$

  donde p es la probabilidad de mantener un parámetro y $1 - p$ de hacerlo 0
- Entonces, durante el forward, se calcula

$$\tilde{\mathbf{h}} = \frac{\mathbf{m} \odot \mathbf{h}}{p}$$

  donde $\odot$ es el producto componente a componente
- En inferencia (fuera del entrenamiento), se desactiva el dropout

Técnicas para evitar overfitting: conclusiones

Takeaways

- Hemos visto tres formas de controlar overfitting: early stopping/monitorización del loss, regularizaciones L_i y dropout
- La monitorización del error de validación conviene hacerla siempre si tenemos suficientes datos de entrenamiento
- Las regularizaciones L_i fuerza a que los parámetros sean pequeños
- El dropout elimina de manera aleatoria salidas de las capas para evitar el sobreajuste
- Las tres técnicas suelen usarse juntas para mejorar la generalización de la red

Optimizadores

Optimizadores

Optimizadores

En el contexto de redes, un optimizador es el algoritmo que decide cómo se actualizan los parámetros del modelo para minimizar la función de pérdida.

- Hemos visto el básico, el SGD:

$$\theta \leftarrow \theta - \lambda \nabla L_B(\theta)$$

- Sin embargo existen muchos: SGD + momentum, RMSProp, Adam, etc.
- Por ejemplo, SGD + momentum sería algo como esto

$$\begin{aligned} v_t &= \beta v_{t-1} + (1-\beta)\nabla L_B(\theta_t) \\ \theta_{t+1} &= \theta_t - \lambda v_t \end{aligned}$$

Optimizadores

- En la práctica el que más se usa es el Adam
- Adam funciona bien en muchos escenarios
- Adam combina varias técnicas: momentum y RMSProp
- Ante la duda, usad Adam y ajustar principalmente (por no decir únicamente) el learning rate λ
- No vamos a entrar en detalles de este optimizador, ni de ningún otro
- Para explicar los optimizadores necesitaría otro curso entero 😅

Takeaway

Adam y optimizadores similares son extensiones de SGD diseñadas para mejorar su estabilidad y velocidad de convergencia. Pero, al final, lo importante es entender el SGD.

Ejercicios

- Entrenamiento y monitorización del loss
- Loss de validación
- Implementar el early stopping
- Evaluación en test
- Implementación de dropout o L_i

Pytorch

Conceptos básicos

¿Qué es PyTorch?

Definición

PyTorch es una librería para construir y entrenar **modelos matemáticos diferenciables**, especialmente redes neuronales, de forma flexible y eficiente.

Se apoya en dos ideas fundamentales:

- **Tensores**: estructuras de datos n–dimensionales.
- **Autograd**: cálculo automático de derivadas.

Filosofía *define-by-run*

El grafo computacional se construye **dinámicamente** al ejecutar el código.

Tensores: el objeto fundamental

¿Qué es un tensor?

Un tensor es un array n-dimensional que:

- Puede vivir en CPU o GPU
- Soporta operaciones vectorizadas
- Puede participar en derivación automática

Ejemplos típicos de tensores y dimensiones:

- Escalar: $(\ )$
- Vector: (d)
- Batch de datos: (B, d)
- Imagen: (C, H, W)

Mensaje importante

En deep learning, **todo son tensores**: datos, pesos y gradientes.

shapes

En PyTorch, las dimensiones o `shapes` importan tanto como los valores. Un operador en PyTorch puede cambiar la dimensión.
Ejemplo típico en una capa lineal:

$$X \in \mathbb{R}^{B \times d}, \quad W \in \mathbb{R}^{d \times k}, \quad b \in \mathbb{R}^{k}$$

$$XW + b \in \mathbb{R}^{B \times k}$$

Ojo!

El shape mismatch es un error muy común. Antes de programar, dibuja las `shapes`.

CPU vs GPU: el concepto de `device`

Cada tensor vive en un **dispositivo**:

- `cpu`
- `cuda` (GPU)

Ojo!

`Expected all tensors to be on the same device` es un error muy típico.

Interpretación

PyTorch no mueve datos automáticamente entre dispositivos.

Idea clave

Modelo y datos deben vivir **siempre** en el mismo device.

`requires_grad`: cuándo hay aprendizaje

Un tensor puede:

- Ser solo datos
- O participar en la optimización

Si un tensor tiene:

```
requires_grad = True
```

entonces PyTorch:

- Guarda las operaciones que lo generan
- Permite calcular derivadas respecto a él

Idea clave

Solo los parámetros del modelo necesitan gradiente.

Autograd

Autograd construye un **grafo computacional dinámico** (mientras se ejecuta el código, el forward):

- Cada operación es un nodo
- Cada tensor resultado conoce cómo se ha calculado

Cuando llamamos:

```
loss.backward()
```

Ocurre el backpropagation:

- Se recorre el grafo hacia atrás
- Se aplica la regla de la cadena
- Se calculan gradientes $\partial L / \partial \theta$

Definición de redes

Red neuronal en PyTorch

Vimos al principio de esta sesión que una red neuronal es algo como esto:

$$f_\theta : \mathbb{R}^n \to \mathbb{R}^m$$

Donde:

- θ son los parámetros
- f_θ es una función computable

En PyTorch esa f_θ se define en una clase Python...

Red neuronal en PyTorch

Todo modelo en PyTorch hereda de `nn.Module` (es la clase base de PyTorch para definir modelos y capas entrenables). Contiene dos partes fundamentales:

- `__init__`: definición de capas
- `forward`: definición del flujo de datos

Interpretación

- `__init__` define los θ
- `forward` define la **fórmula matemática** f_θ del modelo.

Mensaje

Tú defines f_θ; PyTorch se encarga de derivarla.

Ejemplo: MLP simple para Iris

Contexto del problema

Queremos clasificar flores del dataset Iris a partir de sus características morfológicas.

Ejemplo: MLP simple para Iris

Contexto del problema

Queremos clasificar flores del dataset Iris a partir de sus características morfológicas.

Cada muestra es un vector:

$$x = (\text{sepal length}, \text{sepal width}, \text{petal length}, \text{petal width}) \in \mathbb{R}^4$$

Objetivo:

$$f_\theta : \mathbb{R}^4 \longrightarrow \mathbb{R}^3$$

donde las i corresponde puntuación (logits) de x con respecto a la clase i.

Nota

En las primeras diapositivas vimos que una red que daba como salida probabilidades al aplicarle la softmax. Por conveniencia, vamos a eliminar la parte de la softmax de la arquitectura para ser consistente con PyTorch. En este framework, el softmax no forma parte de la arquitectura y se integra con la loss.

Arquitectura de la red

Arquitectura concreta:

$$\mathbb{R}^4 \xrightarrow{\text{Linear}(4,16)} \mathbb{R}^{16} \xrightarrow{\text{ReLU}} \mathbb{R}^{16} \xrightarrow{\text{Linear}(16,3)} \mathbb{R}^3$$

- Se aplica una capa lineal
- Se aplica relu
- Se aplica una capa lineal, da como resultado los logits

La MLP como composición de funciones

Formalmente, la red define la función:

$$f_\theta(x) = f_3\big(f_2(f_1(x))\big)$$

donde:

$$\begin{aligned} f_1(x) &= W_1 x + b_1 \\ f_2(h) &= \mathrm{ReLU}(h) \\ f_3(h) &= W_2 h + b_2 \end{aligned}$$

- Parámetros: $W_1 \in \mathbb{R}^{16\times 4}$, $b_1 \in \mathbb{R}^{16\times 1}$, $W_2 \in \mathbb{R}^{3\times 16}$, $b_2 \in \mathbb{R}^{3\times 1}$
- Número de parámetros/dimensiones: $16 \times 4 + 16 + 16 \times 3 + 3 = 131$

Correspondencia directa en PyTorch

Cada bloque matemático tiene su equivalente directo:

- $f_1(x) = W_1 x + b_1 \quad \leftrightarrow$ `nn.Linear(4,16)`
- $f_2(x) = \mathrm{ReLU}(x) \quad \leftrightarrow$ `nn.ReLU()`
- $f_3(x) = W_2 x + b_2 \quad \leftrightarrow$ `nn.Linear(16,3)`

Takeaway

Implementar una red en PyTorch es traducir esta composición matemática a código. Hay un montón de operadores ya implementados listos para usar 😄: `https://docs.pytorch.org/docs/stable/index.html`.

MLP Iris (PyTorch)

```python
import torch
import torch.nn as nn

# importante que la case herede de nn.Module
# nn.Module es la clase base de PyTorch para definir modelos y capas entrenables
class IrisMLP(nn.Module):
    def __init__(self, hidden_dim: int = 16):
        super().__init__()
        self.fc1 = nn.Linear(4, hidden_dim)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden_dim, 3)
# importante: el forward asume que recibe como entrada un batch de datos
# en el ejemplo (B, 4)
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        h = self.relu(self.fc1(x))   # (B, 16)
        logits = self.fc2(h)         # (B, 3)
        return logits
```

Bucle de entrenamiento

Entrenamiento de un red

1. Inicializar los parámetros θ de la red.
2. Repetir durante un número de **epochs**:
   - 2.1 Barajar el conjunto de entrenamiento.
   - 2.2 Dividir el dataset en **batches**.
   - 2.3 Para cada batch B:
     - 2.3.1 Forward pass: calcular la salida de la red.
     - 2.3.2 Calcular la loss del batch.
     - 2.3.3 Backward pass.
     - 2.3.4 Actualizar los parámetros

Entrenamiento de un red

1. Inicializar los parámetros θ de la red. `Instanciar la red`
2. Repetir durante un número de **epochs**: `Bucle manual`
   - 2.1 Barajar el conjunto de entrenamiento. `Dataloader`
   - 2.2 Dividir el dataset en **batches**. `Dataloader`
   - 2.3 Para cada batch B: `Dataloader`
     - 2.3.1 Forward pass: calcular la salida de la red. `Llamada a forward()`
     - 2.3.2 Calcular la loss del batch. `PyTorch losses`
     - 2.3.3 Backward pass. `Llamada a backward()`
     - 2.3.4 Actualizar los parámetros `Optimizador`

Ya lo hemos visto casi todo, solo falta ver:

- PyTorch Datasets y Dataloaders
- PyTorch losses
- PyTorch optimizers

PyTorch Datasets

¿Qué es un Dataset?

Un `Dataset` define:

- qué es un ejemplo de datos
- cómo acceder al ejemplo i
- cuántos ejemplos hay

Formalmente, un `Dataset` implementa:

- `__len__()` $\rightarrow$ número de ejemplos
- `__getitem__(i)` $\rightarrow (x_i, y_i)$

Ejemplo conceptual:

$$(x_i, y_i) = \texttt{dataset}[i]$$

Idea clave

El `Dataset` define **qué datos tenemos**, pero no cómo se recorren.

PyTorch Datasets: ejemplo

```python
import torch
from torch.utils.data import Dataset
from sklearn.datasets import load_iris

class IrisDataset(Dataset):
    def __init__(self):
        data = load_iris()
        self.X = torch.tensor(
            data.data, dtype=torch.float32
        )  # (150, 4)
        self.y = torch.tensor(
            data.target, dtype=torch.long
        )  # (150,)

    def __len__(self):
        return self.X.shape[0]

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]
```

PyTorch DataLoaders

¿Qué es un DataLoader?

Un `DataLoader` se encarga de:

- barajar los datos (`shuffle`)
- dividirlos en **batches**
- iterar sobre el dataset

Uso típico:

```
train_loader = DataLoader(
    dataset,
    batch_size=32,
    shuffle=True
)
```

Dentro del bucle de entrenamiento:

```
for x_B, y_B in train_loader:
    [...]
```

Idea clave

El `DataLoader` no contiene datos: **itera sobre un Dataset**.

Pytorch losses

En PyTorch, una loss es un objeto:

- recibe predicciones y etiquetas
- devuelve un escalar
- es derivable
- sobre el resultado de aplicar el loss, se hace el backward

Ejemplo clasificación:

```
criterion = nn.CrossEntropyLoss() # aplica el softmax y calcula el error
loss = criterion(logits, y)
```

Ejemplo regresión:

```
criterion = nn.MSELoss()
loss = criterion(y_pred, y)
```

Optimizadores en PyTorch

Problema que resuelven

Una vez calculados los gradientes, necesitamos actualizar los parámetros θ.

$$\theta \leftarrow \theta - \lambda \nabla L_B(\theta)$$

En PyTorch, un optimizador:

- recibe los parámetros del modelo
- usa los gradientes calculados por `backward()`
- actualiza θ

Ejemplo típico:

```
optimizer = torch.optim.Adam(
    model.parameters(), lr=1e-3
)
```

Paso clave en el entrenamiento:

```
optimizer.step()
```

Importante

El optimizador no calcula gradientes: solo los usa para mover los parámetros.

Ejemplo del entrenamiento final

```
dataset = IrisDataset()
train_loader = DataLoader(dataset, batch_size=32, shuffle=True)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = IrisMLP(hidden_dim=16).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

num_epochs = 100
for epoch in range(1, num_epochs + 1):
    model.train()
    running_loss = 0.0
    n_seen = 0
    for xB, yB in train_loader:
        xB, yB = xB.to(device), yB.to(device)
        optimizer.zero_grad()       # reset gradientes
        logits = model(xB)          # forward
        loss = criterion(logits, yB)
        loss.backward()             # backward
        optimizer.step()            # update
        running_loss += loss.item() * xB.size(0)
        n_seen += xB.size(0)
    epoch_loss = running_loss / n_seen
    print(f"Epoch {epoch:03d} | train_loss = {epoch_loss:.4f}")
```

Ejercicio final

Ejercicio

Escoge un dataset de regresión o clasificación (el que tu quieras) y diseña una red neuronal (de varias capas) para clasificarlo. El algoritmo de entrenamiento debe incluir algún tipo de monitorización del conjunto de validación y algún método de regularización (dropout, L_1 o L_2). El modelo debe evaluarse en un conjunto de test.